PORTFOLIOPRO — COMPLETE SOURCE CODE COMPILATION

Backend (Spring Boot 3 / Java 17) & Frontend (Angular 19 / TypeScript)
Candidate: Samarth Singh | HCL Career Shaper COE Submission
Live App: **https://portfolio-pro-ten-lilac.vercel.app** | GitHub: **https://github.com/samarthsingh1507/Portfolio-Pro**

Backend — Spring Security 6 Configuration

File: backend/src/main/java/com/portfoliopro/config/SecurityConfig.java

```
package com.portfoliopro.config;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.portfoliopro.dto.ErrorResponse;
import com.portfoliopro.security.CustomUserDetailsService;
import com.portfoliopro.security.JwtAuthenticationFilter;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;
    private final CustomUserDetailsService userDetailsService;
    private final ObjectMapper objectMapper;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter,
                          CustomUserDetailsService userDetailsService,
                          ObjectMapper objectMapper) {
        this.jwtAuthFilter = jwtAuthFilter;
        this.userDetailsService = userDetailsService;
        this.objectMapper = objectMapper;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
        authProvider.setUserDetailsService(userDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(cors -> {})
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/api/health/**", "/api/stocks/**", "/api/analysis/**",
"/api/market/**").permitAll()
                .anyRequest().authenticated()
            )
            .exceptionHandling(exceptions -> exceptions
```

```
                .authenticationEntryPoint((request, response, authException) -> {
                    response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
                    response.setContentType(MediaType.APPLICATION_JSON_VALUE);
                    ErrorResponse errorResponse = new ErrorResponse(
                        HttpStatus.UNAUTHORIZED.value(),
                        "Unauthorized",
                        "Authentication is required to access this resource",
                        request.getRequestURI()
                    );
                    response.getWriter().write(objectMapper.writeValueAsString(errorResponse));
                })
                .accessDeniedHandler((request, response, accessDeniedException) -> {
                    response.setStatus(HttpServletResponse.SC_FORBIDDEN);
                    response.setContentType(MediaType.APPLICATION_JSON_VALUE);
                    ErrorResponse errorResponse = new ErrorResponse(
                        HttpStatus.FORBIDDEN.value(),
                        "Forbidden",
                        "You do not have permission to access this resource",
                        request.getRequestURI()
                    );
                    response.getWriter().write(objectMapper.writeValueAsString(errorResponse));
                })
            )
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Backend — Dynamic CORS Configuration

File: backend/src/main/java/com/portfoliopro/config/CorsConfig.java

```
package com.portfoliopro.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.filter.CorsFilter;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

@Configuration
public class CorsConfig {

    @Value("${app.cors.allowed-origins:http://localhost:4200,http://127.0.0.1:4200}")
    private String allowedOrigins;

    @Bean
    public CorsFilter corsFilter() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowCredentials(true);

        List<String> origins = Arrays.stream(allowedOrigins.split(","))
                .map(String::trim)
                .filter(origin -> !origin.isEmpty())
                .collect(Collectors.toList());

        config.setAllowedOriginPatterns(origins);
        config.setAllowedHeaders(Arrays.asList("Origin", "Content-Type", "Accept", "Authorization", "X-Requested-With"));
        config.setExposedHeaders(Arrays.asList("Authorization", "Content-Disposition"));
        config.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH"));

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return new CorsFilter(source);
    }
}
```

Backend — Stateless JWT Authentication Filter

File: backend/src/main/java/com/portfoliopro/security/JwtAuthenticationFilter.java

```
package com.portfoliopro.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.lang.NonNull;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final CustomUserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtService jwtService, CustomUserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
            @NonNull HttpServletRequest request,
            @NonNull HttpServletResponse response,
            @NonNull FilterChain filterChain
    ) throws ServletException, IOException {
        final String authHeader = request.getHeader("Authorization");
        final String jwt;
        final String username;

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        jwt = authHeader.substring(7);
        try {
            username = jwtService.extractUsername(jwt);
            if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
                UserDetails userDetails = this.userDetailsService.loadUserByUsername(username);
                if (jwtService.isTokenValid(jwt, userDetails)) {
                    UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                            userDetails,
                            null,
                            userDetails.getAuthorities()
                    );
                    authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContextHolder.getContext().setAuthentication(authToken);
                }
            }
        } catch (Exception e) {
            // Invalid, malformed or expired JWT token; authentication will not be set
            logger.warn("JWT token validation failed: " + e.getMessage());
        }

        filterChain.doFilter(request, response);
    }
}
```

Backend — Cryptographic JWT Token Service

File: backend/src/main/java/com/portfoliopro/security/JwtService.java

```
package com.portfoliopro.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Service
public class JwtService {

    @Value("${app.jwt.secret}")
    private String secret;

    @Value("${app.jwt.expiration-ms}")
    private long expirationMs;

    private SecretKey getSigningKey() {
        byte[] keyBytes = secret.getBytes(StandardCharsets.UTF_8);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    public String generateToken(UserDetails userDetails) {
        return generateToken(new HashMap<>(), userDetails);
    }

    public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) {
        return Jwts.builder()
                .claims(extraClaims)
                .subject(userDetails.getUsername())
                .issuedAt(new Date(System.currentTimeMillis()))
                .expiration(new Date(System.currentTimeMillis() + expirationMs))
                .signWith(getSigningKey())
                .compact();
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername())) && !isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parser()
                .verifyWith(getSigningKey())
                .build()
```

```
                .parseSignedClaims(token)
                .getPayload();
    }
}
```

Backend — Auth REST Controller

File: backend/src/main/java/com/portfoliopro/controller/AuthController.java

```
package com.portfoliopro.controller;

import com.portfoliopro.dto.ApiResponse;
import com.portfoliopro.dto.LoginRequest;
import com.portfoliopro.dto.LoginResponse;
import com.portfoliopro.dto.RegisterRequest;
import com.portfoliopro.service.AuthService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    public ResponseEntity<ApiResponse> register(@Valid @RequestBody RegisterRequest request) {
        ApiResponse response = authService.register(request);
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(@Valid @RequestBody LoginRequest request) {
        LoginResponse response = authService.login(request);
        return ResponseEntity.ok(response);
    }
}
```

Backend — Stock Catalog REST Controller

File: backend/src/main/java/com/portfoliopro/controller/MarketController.java

```
package com.portfoliopro.controller;

import com.portfoliopro.dto.StockDetailResponse;
import com.portfoliopro.dto.StockResponse;
import com.portfoliopro.service.MarketService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.security.Principal;
import java.util.List;

@RestController
@RequestMapping("/api/stocks")
public class MarketController {

    private final MarketService marketService;

    public MarketController(MarketService marketService) {
        this.marketService = marketService;
    }

    @GetMapping
    public ResponseEntity<List<StockResponse>> getAllStocks() {
        List<StockResponse> stocks = marketService.getAllStocks();
        return ResponseEntity.ok(stocks);
    }

    @GetMapping(value = {"/{symbol}", "/{symbol1}/{symbol2}"})
    public ResponseEntity<StockDetailResponse> getStockDetail(@PathVariable(required = false) String symbol,
                                                              @PathVariable(required = false) String symbol1,
                                                              @PathVariable(required = false) String symbol2,
                                                              Principal principal) {
        String targetSymbol = symbol;
        if (targetSymbol == null && symbol1 != null && symbol2 != null) {
            targetSymbol = symbol1 + "/" + symbol2;
        }
        String username = principal != null ? principal.getName() : null;
        StockDetailResponse detail = marketService.getStockDetail(targetSymbol, username);
        return ResponseEntity.ok(detail);
    }
}
```

Backend — Trade Execution REST Controller

File: backend/src/main/java/com/portfoliopro/controller/TradeController.java

```
package com.portfoliopro.controller;

import com.portfoliopro.dto.OrderRequest;
import com.portfoliopro.dto.OrderResponse;
import com.portfoliopro.service.TradeService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.security.Principal;
import java.util.List;

@RestController
@RequestMapping("/api/orders")
public class TradeController {

    private final TradeService tradeService;

    public TradeController(TradeService tradeService) {
        this.tradeService = tradeService;
    }

    @PostMapping
    public ResponseEntity<OrderResponse> placeOrder(@Valid @RequestBody OrderRequest request, Principal principal) {
        OrderResponse response = tradeService.executeOrder(request, principal.getName());
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }

    @GetMapping
    public ResponseEntity<List<OrderResponse>> getOrderHistory(Principal principal) {
        List<OrderResponse> orders = tradeService.getOrderHistory(principal.getName());
        return ResponseEntity.ok(orders);
    }
}
```

Backend — Mark-to-Market Portfolio Controller

File: backend/src/main/java/com/portfoliopro/controller/PortfolioController.java

```
package com.portfoliopro.controller;

import com.portfoliopro.dto.PortfolioAnalyticsResponse;
import com.portfoliopro.dto.PortfolioResponse;
import com.portfoliopro.dto.PortfolioSummaryResponse;
import com.portfoliopro.service.PortfolioService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.security.Principal;

@RestController
@RequestMapping("/api/portfolio")
public class PortfolioController {

    private final PortfolioService portfolioService;

    public PortfolioController(PortfolioService portfolioService) {
        this.portfolioService = portfolioService;
    }

    @GetMapping
    public ResponseEntity<PortfolioResponse> getPortfolio(Principal principal) {
        PortfolioResponse portfolio = portfolioService.getPortfolio(principal.getName());
        return ResponseEntity.ok(portfolio);
    }

    @GetMapping("/summary")
    public ResponseEntity<PortfolioSummaryResponse> getPortfolioSummary(Principal principal) {
        PortfolioSummaryResponse summary = portfolioService.getPortfolioSummary(principal.getName());
        return ResponseEntity.ok(summary);
    }

    @GetMapping("/analytics")
    public ResponseEntity<PortfolioAnalyticsResponse> getPortfolioAnalytics(Principal principal) {
        PortfolioAnalyticsResponse analytics = portfolioService.getPortfolioAnalytics(principal.getName());
        return ResponseEntity.ok(analytics);
    }
}
```

Backend — Technical Analysis Controller

File: backend/src/main/java/com/portfoliopro/controller/AnalysisController.java

```
package com.portfoliopro.controller;

import com.portfoliopro.dto.FundamentalAnalysisResponse;
import com.portfoliopro.dto.TechnicalAnalysisResponse;
import com.portfoliopro.service.AnalysisService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/analysis")
public class AnalysisController {

    private final AnalysisService analysisService;

    public AnalysisController(AnalysisService analysisService) {
        this.analysisService = analysisService;
    }

    @GetMapping(value = {"/technical/{symbol}", "/technical/{symbol1}/{symbol2}"})
    public ResponseEntity<TechnicalAnalysisResponse> getTechnicalAnalysis(@PathVariable(required = false) String symbol,
                                                                           @PathVariable(required = false) String symbol1,
                                                                           @PathVariable(required = false) String symbol2)
{
        String targetSymbol = symbol;
        if (targetSymbol == null && symbol1 != null && symbol2 != null) {
            targetSymbol = symbol1 + "/" + symbol2;
        }
        TechnicalAnalysisResponse response = analysisService.getTechnicalAnalysis(targetSymbol);
        return ResponseEntity.ok(response);
    }

    @GetMapping(value = {"/fundamental/{symbol}", "/fundamental/{symbol1}/{symbol2}"})
    public ResponseEntity<FundamentalAnalysisResponse> getFundamentalAnalysis(@PathVariable(required = false) String
symbol,
                                                                               @PathVariable(required = false) String
symbol1,
                                                                               @PathVariable(required = false) String
symbol2) {
        String targetSymbol = symbol;
        if (targetSymbol == null && symbol1 != null && symbol2 != null) {
            targetSymbol = symbol1 + "/" + symbol2;
        }
        FundamentalAnalysisResponse response = analysisService.getFundamentalAnalysis(targetSymbol);
        return ResponseEntity.ok(response);
    }
}
```

Backend — Order Execution & Cost Basis Service

File: backend/src/main/java/com/portfoliopro/service/TradeService.java

```
package com.portfoliopro.service;

import com.portfoliopro.dto.OrderRequest;
import com.portfoliopro.dto.OrderResponse;
import com.portfoliopro.entity.Holding;
import com.portfoliopro.entity.Order;
import com.portfoliopro.entity.Stock;
import com.portfoliopro.entity.User;
import com.portfoliopro.exception.BadRequestException;
import com.portfoliopro.exception.ResourceNotFoundException;
import com.portfoliopro.repository.HoldingRepository;
import com.portfoliopro.repository.OrderRepository;
import com.portfoliopro.repository.StockRepository;
import com.portfoliopro.repository.UserRepository;
import com.portfoliopro.service.valuation.PortfolioValuationService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

@Service
public class TradeService {

    private final OrderRepository orderRepository;
    private final HoldingRepository holdingRepository;
    private final StockRepository stockRepository;
    private final UserRepository userRepository;
    private final PortfolioValuationService portfolioValuationService;

    public TradeService(OrderRepository orderRepository,
                        HoldingRepository holdingRepository,
                        StockRepository stockRepository,
                        UserRepository userRepository,
                        PortfolioValuationService portfolioValuationService) {
        this.orderRepository = orderRepository;
        this.holdingRepository = holdingRepository;
        this.stockRepository = stockRepository;
        this.userRepository = userRepository;
        this.portfolioValuationService = portfolioValuationService;
    }

    @Transactional
    public OrderResponse executeOrder(OrderRequest request, String username) {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new ResourceNotFoundException("User not found with username: " + username));

        Stock stock = stockRepository.findById(request.getStockId())
                .orElseThrow(() -> new ResourceNotFoundException("Stock not found with id: " + request.getStockId()));

        if (request.getQuantity() == null || request.getQuantity() <= 0) {
            throw new BadRequestException("Order quantity must be greater than zero");
        }

        if (request.getPrice() == null || request.getPrice().compareTo(BigDecimal.ZERO) <= 0) {
            throw new BadRequestException("Order price must be greater than zero");
        }

        String orderType = request.getType().trim().toUpperCase();
        if (!"BUY".equals(orderType) && !"SELL".equals(orderType)) {
            throw new BadRequestException("Invalid order type: " + request.getType() + ". Must be BUY or SELL");
        }

        if ("BUY".equals(orderType)) {
            executeBuyOrder(user, stock, request.getQuantity(), request.getPrice());
        } else {
            executeSellOrder(user, stock, request.getQuantity(), request.getPrice());
```

```
        }

        // Create executed order record
        Order order = new Order(
                user,
                stock,
                orderType,
                request.getQuantity(),
                request.getPrice(),
                "EXECUTED"
        );
        order = orderRepository.save(order);

        BigDecimal totalValue = request.getPrice().multiply(BigDecimal.valueOf(request.getQuantity())).setScale(2,
RoundingMode.HALF_UP);

        return new OrderResponse(
                order.getId(),
                stock.getId(),
                stock.getSymbol(),
                stock.getCompanyName(),
                order.getType(),
                order.getQuantity(),
                order.getPrice(),
                totalValue,
                order.getStatus(),
                order.getCreatedAt()
        );
    }

    private void executeBuyOrder(User user, Stock stock, Integer quantity, BigDecimal price) {
        Optional<Holding> existingHoldingOpt = holdingRepository.findByUserAndStock(user, stock);

        if (existingHoldingOpt.isEmpty()) {
            Holding newHolding = new Holding(user, stock, quantity, price.setScale(2, RoundingMode.HALF_UP));
            holdingRepository.save(newHolding);
        } else {
            Holding holding = existingHoldingOpt.get();
            int oldQuantity = holding.getQuantity();
            BigDecimal oldAvg = holding.getAvgBuyPrice();

            int newQuantity = oldQuantity + quantity;
            BigDecimal newAverage = portfolioValuationService.calculateNewAverageBuyPrice(
                    oldQuantity, oldAvg, quantity, price);

            holding.setQuantity(newQuantity);
            holding.setAvgBuyPrice(newAverage);
            holdingRepository.save(holding);
        }
    }

    private void executeSellOrder(User user, Stock stock, Integer quantity, BigDecimal price) {
        Optional<Holding> existingHoldingOpt = holdingRepository.findByUserAndStock(user, stock);

        if (existingHoldingOpt.isEmpty() || existingHoldingOpt.get().getQuantity() <= 0) {
            throw new BadRequestException("You do not own any shares of " + stock.getSymbol());
        }

        Holding holding = existingHoldingOpt.get();
        if (quantity > holding.getQuantity()) {
            throw new BadRequestException("Insufficient holding quantity. You own " + holding.getQuantity() +
                    " shares, but attempted to sell " + quantity);
        }

        int remainingQuantity = holding.getQuantity() - quantity;
        if (remainingQuantity == 0) {
            holdingRepository.delete(holding);
        } else {
            holding.setQuantity(remainingQuantity);
            holdingRepository.save(holding);
        }
    }

    @Transactional(readOnly = true)
```

```
    public List<OrderResponse> getOrderHistory(String username) {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new ResourceNotFoundException("User not found with username: " + username));

        return orderRepository.findByUserOrderByCreatedAtDesc(user).stream()
                .map(this::mapToOrderResponse)
                .collect(Collectors.toList());
    }

    private OrderResponse mapToOrderResponse(Order order) {
        Stock stock = order.getStock();
        BigDecimal totalValue = order.getPrice().multiply(BigDecimal.valueOf(order.getQuantity())).setScale(2,
RoundingMode.HALF_UP);

        return new OrderResponse(
                order.getId(),
                stock.getId(),
                stock.getSymbol(),
                stock.getCompanyName(),
                order.getType(),
                order.getQuantity(),
                order.getPrice(),
                totalValue,
                order.getStatus(),
                order.getCreatedAt()
        );
    }
}
```

Backend — Portfolio Mark-to-Market Service

File: backend/src/main/java/com/portfoliopro/service/PortfolioService.java

```
package com.portfoliopro.service;

import com.portfoliopro.dto.AllocationItemDto;
import com.portfoliopro.dto.HoldingResponse;
import com.portfoliopro.dto.PortfolioAnalyticsResponse;
import com.portfoliopro.dto.PortfolioResponse;
import com.portfoliopro.dto.PortfolioSummaryResponse;
import com.portfoliopro.entity.Holding;
import com.portfoliopro.entity.User;
import com.portfoliopro.exception.ResourceNotFoundException;
import com.portfoliopro.repository.HoldingRepository;
import com.portfoliopro.repository.UserRepository;
import com.portfoliopro.service.valuation.PortfolioValuationService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;

@Service
public class PortfolioService {

    private final HoldingRepository holdingRepository;
    private final UserRepository userRepository;
    private final PortfolioValuationService portfolioValuationService;

    public PortfolioService(HoldingRepository holdingRepository,
                            UserRepository userRepository,
                            PortfolioValuationService portfolioValuationService) {
        this.holdingRepository = holdingRepository;
        this.userRepository = userRepository;
        this.portfolioValuationService = portfolioValuationService;
    }

    @Transactional(readOnly = true)
    public PortfolioResponse getPortfolio(String username) {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new ResourceNotFoundException("User not found with username: " + username));

        List<Holding> holdings = holdingRepository.findByUserOrderByStockSymbolAsc(user);
        BigDecimal availableCash = BigDecimal.ZERO.setScale(2, RoundingMode.HALF_UP);

        return portfolioValuationService.valuatePortfolio(holdings, availableCash);
    }

    @Transactional(readOnly = true)
    public PortfolioSummaryResponse getPortfolioSummary(String username) {
        PortfolioResponse portfolio = getPortfolio(username);
        return new PortfolioSummaryResponse(
                portfolio.getTotalInvested(),
                portfolio.getCurrentValue(),
                portfolio.getTotalProfitLoss(),
                portfolio.getProfitLossPercent(),
                portfolio.getAvailableCash(),
                portfolio.getMarketDataAvailable(),
                portfolio.getValuationStatus()
        );
    }

    @Transactional(readOnly = true)
    public PortfolioAnalyticsResponse getPortfolioAnalytics(String username) {
        PortfolioResponse portfolio = getPortfolio(username);

        BigDecimal totalInvested = portfolio.getTotalInvested();
        BigDecimal currentValue = portfolio.getCurrentValue();
        BigDecimal totalProfitLoss = portfolio.getTotalProfitLoss();
        BigDecimal profitLossPercent = portfolio.getProfitLossPercent();
        List<HoldingResponse> holdings = portfolio.getHoldings();
```

```
        int numberOfHoldings = holdings.size();

        List<AllocationItemDto> allocation = new ArrayList<>();
        if (currentValue != null && currentValue.compareTo(BigDecimal.ZERO) > 0) {
            for (HoldingResponse h : holdings) {
                if (h.getCurrentValue() != null) {
                    BigDecimal pct = h.getCurrentValue()
                            .multiply(BigDecimal.valueOf(100))
                            .divide(currentValue, 2, RoundingMode.HALF_UP);
                    allocation.add(new AllocationItemDto(h.getSymbol(), pct, h.getCompanyName(), h.getCurrentValue()));
                } else {
                    allocation.add(new AllocationItemDto(h.getSymbol(), BigDecimal.ZERO.setScale(2,
RoundingMode.HALF_UP), h.getCompanyName(), BigDecimal.ZERO.setScale(2, RoundingMode.HALF_UP)));
                }
            }
        } else {
            for (HoldingResponse h : holdings) {
                allocation.add(new AllocationItemDto(h.getSymbol(), BigDecimal.ZERO.setScale(2, RoundingMode.HALF_UP),
h.getCompanyName(), BigDecimal.ZERO.setScale(2, RoundingMode.HALF_UP)));
            }
        }

        String diversification;
        String riskIndicator;

        if (numberOfHoldings == 0) {
            diversification = "No Holdings";
            riskIndicator = "No Exposure";
        } else if (numberOfHoldings == 1) {
            diversification = "Low Diversification";
            riskIndicator = "High Concentration";
        } else if (numberOfHoldings <= 3) {
            diversification = "Moderate Diversification";
            riskIndicator = "Moderate Concentration";
        } else {
            diversification = "Higher Diversification";
            riskIndicator = "Lower Concentration";
        }

        return new PortfolioAnalyticsResponse(
                totalInvested,
                currentValue,
                totalProfitLoss,
                profitLossPercent,
                numberOfHoldings,
                allocation,
                diversification,
                riskIndicator
        );
    }
}
```

Backend — Quantitative Technical Analysis

File: backend/src/main/java/com/portfoliopro/service/AnalysisService.java

```
package com.portfoliopro.service;

import com.portfoliopro.dto.FundamentalAnalysisResponse;
import com.portfoliopro.dto.PriceHistoryItemDto;
import com.portfoliopro.dto.TechnicalAnalysisResponse;
import com.portfoliopro.entity.PriceHistory;
import com.portfoliopro.entity.Stock;
import com.portfoliopro.exception.ResourceNotFoundException;
import com.portfoliopro.repository.PriceHistoryRepository;
import com.portfoliopro.repository.StockRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class AnalysisService {

    private final StockRepository stockRepository;
    private final PriceHistoryRepository priceHistoryRepository;
    private final com.portfoliopro.service.market.MarketDataService marketDataService;
    private final MarketService marketService;

    public AnalysisService(StockRepository stockRepository,
                           PriceHistoryRepository priceHistoryRepository,
                           com.portfoliopro.service.market.MarketDataService marketDataService,
                           MarketService marketService) {
        this.stockRepository = stockRepository;
        this.priceHistoryRepository = priceHistoryRepository;
        this.marketDataService = marketDataService;
        this.marketService = marketService;
    }

    @Transactional
    public TechnicalAnalysisResponse getTechnicalAnalysis(String symbol) {
        Stock stock = stockRepository.findBySymbolIgnoreCase(symbol.trim())
                .orElseGet(() -> marketService.findOrCreateStock(symbol));

        BigDecimal currentPrice = stock.getPrice();
        if (marketDataService != null) {
            try {
                com.portfoliopro.dto.market.MarketQuoteDto quote = marketDataService.getQuote(stock.getSymbol());
                if (quote != null && quote.getPrice() != null) {
                    currentPrice = quote.getPrice();
                }
            } catch (Exception ignored) {
                // Fallback gracefully to stock.getPrice()
            }
        }

        List<PriceHistoryItemDto> priceHistoryDtos = generate5YearHistory(stock, currentPrice);

        BigDecimal ma20 = calculateMovingAverageFromDtos(priceHistoryDtos, 20);
        BigDecimal ma50 = calculateMovingAverageFromDtos(priceHistoryDtos, 50);
        BigDecimal rsi14 = calculateRsiFromDtos(priceHistoryDtos, 14);

        return new TechnicalAnalysisResponse(
                stock.getSymbol(),
                currentPrice,
                ma20,
                ma50,
                rsi14,
                priceHistoryDtos
        );
    }
```

```
    private List<PriceHistoryItemDto> generate5YearHistory(Stock stock, BigDecimal finalPrice) {
        LocalDate end = LocalDate.now();
        LocalDate start = end.minusYears(5);
        List<LocalDate> dates = new ArrayList<>();
        LocalDate curr = start;
        while (!curr.isAfter(end)) {
            if (curr.getDayOfWeek().getValue() <= 5) {
                dates.add(curr);
            }
            curr = curr.plusDays(1);
        }

        int totalPoints = dates.size();
        double target = finalPrice != null ? finalPrice.doubleValue() : (stock.getPrice() != null ?
stock.getPrice().doubleValue() : 100.0);
        String sym = stock.getSymbol() != null ? stock.getSymbol().toUpperCase() : "AAPL";
        boolean isGold = sym.contains("XAU") || sym.equals("GOLD");
        boolean isSilver = sym.contains("XAG") || sym.equals("SILVER");
        boolean isForex = (sym.contains("/") || (stock.getSector() != null &&
stock.getSector().toLowerCase().contains("forex"))) && !isGold && !isSilver;
        int scale = (isForex && target < 20.0) ? 4 : 2;

        double startRatio;
        if (isGold) {
            startRatio = 0.68; // Gold 5-year steady appreciation from ~$1,800 to ~$2,658
        } else if (isSilver) {
            startRatio = 0.58;
        } else if (isForex) {
            startRatio = switch (sym) {
                case "EUR/USD" -> 1.04;
                case "GBP/USD" -> 1.03;
                case "USD/JPY" -> 0.74;
                case "USD/CHF" -> 1.08;
                case "AUD/USD" -> 1.07;
                case "USD/CAD" -> 0.94;
                case "NZD/USD" -> 1.10;
                case "EUR/GBP" -> 1.02;
                default -> 0.98;
            };
        } else {
            startRatio = switch (sym) {
                case "NVDA" -> 0.08;
                case "META" -> 0.38;
                case "GOOGL" -> 0.28;
                case "AAPL" -> 0.36;
                case "MSFT" -> 0.44;
                case "TSLA" -> 0.65;
                case "AMZN" -> 0.64;
                case "JPM" -> 0.43;
                default -> 0.40;
            };
        }

        List<PriceHistoryItemDto> dtoList = new ArrayList<>(totalPoints);
        double startPrice = target * startRatio;
        double currentPriceLevel = startPrice;

        for (int i = 0; i < totalPoints; i++) {
            double progress = (double) i / Math.max(1, totalPoints - 1);
            LocalDate date = dates.get(i);

            // Realistic multi-year macroeconomic wave:
            double macroCycle = isForex
                    ? (Math.sin(progress * Math.PI * 3.5) * 0.05 + Math.cos(progress * 14.0) * 0.012)
                    : (Math.sin(progress * Math.PI * 2.4 - 0.4) * 0.18 + Math.sin(progress * Math.PI * 5.2) * 0.06 +
Math.cos(progress * 19.0) * 0.02);

            double trendBase = startPrice + (target - startPrice) * (isForex ? progress : Math.pow(progress, 1.25));
            double targetTrend = trendBase * (1.0 + macroCycle);

            // Daily volatility creating alternating green / red candle bodies
            double volatility = isForex ? 0.0045 : 0.018;
            double dailyNoise = (Math.sin(i * 2.7) * 0.55 + Math.cos(i * 5.3) * 0.35 + Math.sin(i * 13.1) * 0.25) *
```

```
volatility;

            double openVal = currentPriceLevel;
            double closeVal = (i == totalPoints - 1) ? target : (openVal * (1.0 + dailyNoise) + (targetTrend - openVal) *
0.15);

            // High & Low wicks
            double wickVol = isForex ? 0.0035 : 0.014;
            double wickUp = (Math.abs(Math.sin(i * 3.3)) * 0.7 + 0.3) * wickVol;
            double wickDown = (Math.abs(Math.cos(i * 4.7)) * 0.7 + 0.3) * wickVol;

            double highVal = Math.max(openVal, closeVal) * (1.0 + wickUp);
            double lowVal = Math.min(openVal, closeVal) * (1.0 - wickDown);

            currentPriceLevel = closeVal;

            BigDecimal pointOpen = BigDecimal.valueOf(Math.max(0.0001, openVal)).setScale(scale, RoundingMode.HALF_UP);
            BigDecimal pointHigh = BigDecimal.valueOf(Math.max(0.0001, highVal)).setScale(scale, RoundingMode.HALF_UP);
            BigDecimal pointLow = BigDecimal.valueOf(Math.max(0.0001, lowVal)).setScale(scale, RoundingMode.HALF_UP);
            BigDecimal pointClose = BigDecimal.valueOf(Math.max(0.0001, closeVal)).setScale(scale, RoundingMode.HALF_UP);

            dtoList.add(new PriceHistoryItemDto(date, pointOpen, pointHigh, pointLow, pointClose));
        }

        return dtoList;
    }

    public BigDecimal calculateMovingAverageFromDtos(List<PriceHistoryItemDto> history, int period) {
        if (history == null || history.size() < period || period <= 0) {
            return null;
        }

        int size = history.size();
        BigDecimal sum = BigDecimal.ZERO;
        for (int i = size - period; i < size; i++) {
            sum = sum.add(history.get(i).getClosePrice());
        }

        return sum.divide(BigDecimal.valueOf(period), 2, RoundingMode.HALF_UP);
    }

    public BigDecimal calculateRsiFromDtos(List<PriceHistoryItemDto> history, int period) {
        if (history == null || history.size() <= period || period <= 0) {
            return null;
        }

        int size = history.size();
        double[] prices = new double[size];
        for (int i = 0; i < size; i++) {
            prices[i] = history.get(i).getClosePrice().doubleValue();
        }

        double sumGain = 0.0;
        double sumLoss = 0.0;

        for (int i = 1; i <= period; i++) {
            double change = prices[i] - prices[i - 1];
            if (change > 0) {
                sumGain += change;
            } else if (change < 0) {
                sumLoss += Math.abs(change);
            }
        }

        double avgGain = sumGain / period;
        double avgLoss = sumLoss / period;

        for (int i = period + 1; i < size; i++) {
            double change = prices[i] - prices[i - 1];
            double gain = change > 0 ? change : 0.0;
            double loss = change < 0 ? Math.abs(change) : 0.0;

            avgGain = (avgGain * (period - 1) + gain) / period;
            avgLoss = (avgLoss * (period - 1) + loss) / period;
```

```
        }

        if (avgLoss == 0.0) {
            return BigDecimal.valueOf(100.0).setScale(2, RoundingMode.HALF_UP);
        }

        double rs = avgGain / avgLoss;
        double rsi = 100.0 - (100.0 / (1.0 + rs));

        return BigDecimal.valueOf(rsi).setScale(2, RoundingMode.HALF_UP);
    }

    @Transactional
    public FundamentalAnalysisResponse getFundamentalAnalysis(String symbol) {
        Stock stock = stockRepository.findBySymbolIgnoreCase(symbol.trim())
                .orElseGet(() -> marketService.findOrCreateStock(symbol));

        return new FundamentalAnalysisResponse(
                stock.getSymbol(),
                stock.getCompanyName(),
                stock.getSector(),
                stock.getMarketCap(),
                stock.getEps(),
                stock.getPeRatio()
        );
    }

    /**
     * Calculates Moving Average for the given period using the latest closing prices.
     * Returns null if there is insufficient historical data.
     */
    public BigDecimal calculateMovingAverage(List<PriceHistory> history, int period) {
        if (history == null || history.size() < period || period <= 0) {
            return null;
        }

        int size = history.size();
        BigDecimal sum = BigDecimal.ZERO;
        for (int i = size - period; i < size; i++) {
            sum = sum.add(history.get(i).getClosePrice());
        }

        return sum.divide(BigDecimal.valueOf(period), 2, RoundingMode.HALF_UP);
    }

    /**
     * Calculates the Relative Strength Index (RSI) using standard Wilder's smoothing.
     * Returns null if there are fewer than (period + 1) historical data points.
     */
    public BigDecimal calculateRSI(List<PriceHistory> history, int period) {
        if (history == null || history.size() <= period || period <= 0) {
            return null;
        }

        int size = history.size();
        double[] prices = new double[size];
        for (int i = 0; i < size; i++) {
            prices[i] = history.get(i).getClosePrice().doubleValue();
        }

        // Calculate initial average gain and loss over the first 'period' changes
        double sumGain = 0.0;
        double sumLoss = 0.0;

        for (int i = 1; i <= period; i++) {
            double change = prices[i] - prices[i - 1];
            if (change > 0) {
                sumGain += change;
            } else if (change < 0) {
                sumLoss += Math.abs(change);
            }
        }

        double avgGain = sumGain / period;
```

```
        double avgLoss = sumLoss / period;

        // Apply Wilder's smoothing for subsequent periods up to the latest price
        for (int i = period + 1; i < size; i++) {
            double change = prices[i] - prices[i - 1];
            double gain = change > 0 ? change : 0.0;
            double loss = change < 0 ? Math.abs(change) : 0.0;

            avgGain = (avgGain * (period - 1) + gain) / period;
            avgLoss = (avgLoss * (period - 1) + loss) / period;
        }

        if (avgLoss == 0.0) {
            if (avgGain == 0.0) {
                return BigDecimal.valueOf(50.00).setScale(2, RoundingMode.HALF_UP);
            }
            return BigDecimal.valueOf(100.00).setScale(2, RoundingMode.HALF_UP);
        }

        double rs = avgGain / avgLoss;
        double rsi = 100.0 - (100.0 / (1.0 + rs));

        if (rsi < 0.0) rsi = 0.0;
        if (rsi > 100.0) rsi = 100.0;

        return BigDecimal.valueOf(rsi).setScale(2, RoundingMode.HALF_UP);
    }
}
```

Database — MySQL 8.0 Relational DDL Schema

File: database/schema.sql

```
-- ============================================================
-- PORTFOLIOPRO Database Schema
-- MySQL 8.x Compatible
-- ============================================================

CREATE DATABASE IF NOT EXISTS portfoliopro;
USE portfoliopro;

-- 1. Users Table
CREATE TABLE IF NOT EXISTS users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 2. Stocks Table
CREATE TABLE IF NOT EXISTS stocks (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    symbol VARCHAR(20) NOT NULL UNIQUE,
    company_name VARCHAR(100) NOT NULL,
    price DECIMAL(12, 2) NOT NULL,
    sector VARCHAR(50) NOT NULL,
    market_cap BIGINT NOT NULL,
    eps DECIMAL(10, 2) NOT NULL,
    pe_ratio DECIMAL(10, 2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 3. Price History Table (For technical indicators: MA20, MA50, RSI)
CREATE TABLE IF NOT EXISTS price_history (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    stock_id BIGINT NOT NULL,
    date DATE NOT NULL,
    close_price DECIMAL(12, 2) NOT NULL,
    CONSTRAINT fk_price_history_stock FOREIGN KEY (stock_id) REFERENCES stocks(id) ON DELETE CASCADE,
    CONSTRAINT uq_stock_date UNIQUE (stock_id, date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 4. Watchlist Table
CREATE TABLE IF NOT EXISTS watchlist (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    stock_id BIGINT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_watchlist_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT fk_watchlist_stock FOREIGN KEY (stock_id) REFERENCES stocks(id) ON DELETE CASCADE,
    CONSTRAINT uq_user_stock_watchlist UNIQUE (user_id, stock_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 5. Orders Table
CREATE TABLE IF NOT EXISTS orders (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    stock_id BIGINT NOT NULL,
    type VARCHAR(10) NOT NULL, -- BUY, SELL
    quantity INT NOT NULL,
    price DECIMAL(12, 2) NOT NULL,
    status VARCHAR(20) NOT NULL, -- EXECUTED, REJECTED
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_orders_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT fk_orders_stock FOREIGN KEY (stock_id) REFERENCES stocks(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 6. Holdings Table
CREATE TABLE IF NOT EXISTS holdings (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    stock_id BIGINT NOT NULL,
```

```
    quantity INT NOT NULL,
    avg_buy_price DECIMAL(12, 2) NOT NULL,
    CONSTRAINT fk_holdings_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT fk_holdings_stock FOREIGN KEY (stock_id) REFERENCES stocks(id) ON DELETE RESTRICT,
    CONSTRAINT uq_user_stock_holding UNIQUE (user_id, stock_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Frontend Service — Authentication & Session Manager

File: frontend/src/app/core/services/auth.service.ts

```
import { Injectable, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';
import { Observable, tap } from 'rxjs';
import { environment } from '../../../environments/environment';
import { ApiResponse, LoginRequest, LoginResponse, RegisterRequest, User } from '../../models/user.model';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private readonly TOKEN_KEY = 'portfolio_pro_token';
  private readonly USER_KEY = 'portfolio_pro_username';
  private apiUrl = environment.apiUrl;

  currentUser = signal<string | null>(this.getStoredUsername());

  constructor(private http: HttpClient, private router: Router) {}

  register(data: RegisterRequest): Observable<ApiResponse> {
    return this.http.post<ApiResponse>(`${this.apiUrl}/auth/register`, data);
  }

  login(credentials: LoginRequest): Observable<LoginResponse> {
    return this.http.post<LoginResponse>(`${this.apiUrl}/auth/login`, credentials).pipe(
      tap(response => {
        if (response && response.token) {
          localStorage.setItem(this.TOKEN_KEY, response.token);
          localStorage.setItem(this.USER_KEY, response.username);
          this.currentUser.set(response.username);
        }
      })
    );
  }

  logout(): void {
    localStorage.removeItem(this.TOKEN_KEY);
    localStorage.removeItem(this.USER_KEY);
    this.currentUser.set(null);
    this.router.navigate(['/login']);
  }

  getToken(): string | null {
    return localStorage.getItem(this.TOKEN_KEY);
  }

  getStoredUsername(): string | null {
    return localStorage.getItem(this.USER_KEY);
  }

  isAuthenticated(): boolean {
    const token = this.getToken();
    return !!token;
  }

  getProfile(): Observable<User> {
    return this.http.get<User>(`${this.apiUrl}/users/profile`);
  }
}
```

Frontend Service — Portfolio API Integration

File: frontend/src/app/core/services/portfolio.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../../environments/environment';
import { Portfolio, PortfolioAnalytics } from '../../models/portfolio.model';

@Injectable({
  providedIn: 'root'
})
export class PortfolioService {
  private apiUrl = environment.apiUrl;

  constructor(private http: HttpClient) {}

  getPortfolio(): Observable<Portfolio> {
    return this.http.get<Portfolio>(`${this.apiUrl}/portfolio`);
  }

  getAnalytics(): Observable<PortfolioAnalytics> {
    return this.http.get<PortfolioAnalytics>(`${this.apiUrl}/portfolio/analytics`);
  }
}
```

Frontend Service — Trade Execution API

File: frontend/src/app/core/services/trade.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../../environments/environment';
import { OrderRequest, OrderResponse } from '../../models/order.model';

@Injectable({
  providedIn: 'root'
})
export class TradeService {
  private apiUrl = environment.apiUrl;

  constructor(private http: HttpClient) {}

  placeOrder(order: OrderRequest): Observable<OrderResponse> {
    return this.http.post<OrderResponse>(`${this.apiUrl}/orders`, order);
  }

  getOrders(): Observable<OrderResponse[]> {
    return this.http.get<OrderResponse[]>(`${this.apiUrl}/orders`);
  }
}
```

Frontend Interceptor — Automated Bearer JWT Header Injection

File: frontend/src/app/core/interceptors/auth.interceptor.ts

```
import { HttpErrorResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { AuthService } from '../services/auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const token = authService.getToken();

  let authReq = req;
  if (token) {
    authReq = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });
  }

  return next(authReq).pipe(
    catchError((error: HttpErrorResponse) => {
      if (error.status === 401 && !req.url.includes('/api/auth/')) {
        authService.logout();
      }
      return throwError(() => error);
    })
  );
};
```